

Contents

1 V3.3.1 Cost Model 设计说明：Structure-aware Cycle Correction	1
1.1 1. 一句话理解当前 cost model	2
1.2 2. 候选策略变量	2
1.3 3. 总公式	2
1.4 4. 单 tile 成本：为什么不是简单 load + compute + store？	3
1.5 5. Base cycles：先算一个没有结构修正的基础估计	3
1.6 6. 结构证据到底是什么？	4
1.6.1 6.1 MLIR 静态结构证据	4
1.6.2 6.2 DES 产物结构证据	5
1.6.3 6.3 trace event name/count 结构证据	5
1.7 7. 结构比例如何融合？	5
1.8 8. 从结构比例到 cycle correction factors	6
1.8.1 8.1 每个 factor 的语义	6
1.8.2 8.2 设计边界	7
1.9 9. Corrected component cycles 逐项展开	7
1.9.1 9.1 Load / Store	7
1.9.2 9.2 Cube / Vector / Fix	7
1.9.3 9.3 Cube-vector pipeline	7
1.9.4 9.4 Scalar/control	7
1.9.5 9.5 Workspace	8
1.9.6 9.6 Sync	8
1.10 10. Overlap 如何处理？	8
1.10.1 10.1 Load/store overlap	8
1.10.2 10.2 为什么 overlap confidence 是窄范围？	8
1.11 11. Penalty 的语义	9
1.11.1 11.1 Capacity penalty	9
1.11.2 11.2 Shape penalty	9
1.11.3 11.3 Legality risk penalty	9
1.12 12. V3.3.1 如何利用结构化信息？	9
1.12.1 12.1 例子：一个 scalar-heavy kernel	10
1.13 13. 和 V3.3 的区别	10
1.13.1 13.1 V3.3 的逻辑	10
1.13.2 13.2 V3.3.1 的逻辑	11
1.13.3 13.3 为什么 V3.3.1 更适合汇报？	11
1.14 14. 当前样本的解释方式	11
1.15 15. 当前模型仍然不能证明什么？	12
1.16 16. 推荐的口头表述	12
1.17 17. 文件和报告中应该怎么看	13
1.18 18. 小结	13

1 V3.3.1 Cost Model 设计说明：Structure-aware Cycle Correction

本文档把当前 cost model 从输入、结构证据、分项 cycles、overlap、sync、penalty 到最终 predicted_cycles 完整展开。核心目标只有一个：

估计一个候选策略的 total predicted cycles，用于候选排序和可解释分析。

当前模型不是实机 msprof cycles，也不是 profiling-calibrated hardware model。它是一个 **structure-aware analytical cycle estimation model**：先用硬件参数和候选策略算出基础 cycles，再用 MLIR 与编译产物中的结构证据修正各分项 cycles 的系统误差，最后汇总为总 cycles 估计。

1.1 1. 一句话理解当前 cost model

当前版本不要理解成：

$\text{score} = \text{load} + \text{compute} + \text{store} + \text{sync} + \text{一堆额外加权惩罚}$

而应该理解成：

$\text{predicted_cycles} = \text{sum}(\text{corrected component cycle estimates}) + \text{necessary constraint/risk penalties}$

更直白地说：

第一步：先估计这个策略本来要搬多少数据、算多少 cube/vector、做多少 sync、产生多少 scalar/control 开销。

第二步：再看这个 kernel 的 MLIR/产物结构到底偏 memory、compute、vector、scalar 还是 sync。

第三步：用这些结构证据修正对应分项的 cycles，而不是到处额外加分扣分。

第四步：把修正后的分项 cycles 加起来，得到 total predicted cycles。

1.2 2. 候选策略变量

一个候选策略记作：

$x = (T, M, P, Y)$

其中：

符号	Plan	主要控制什么
T	TilingPlan	tile_m/tile_n/tile_k、block_dim、loop_order、
M	MultiBufferPlan	double buffer、per-buffer multiplier、stage buffer、load/store overlap
P	CVPipelinePlan	cube-vector pipeline、pipeline stage、producer-consumer distance、CV template
Y	SyncPlan	keep existing / graph sync solver、event reuse、sync granularity、sync motion

cost model 的任务是：给每个 x 算一个 $\text{predicted_cycles}(x)$ ，然后搜索器选择 cycles 最低且通过硬件边界检查的候选。

1.3 3. 总公式

当前总成本为：

$$\begin{aligned} T_{\text{total}}(x) = & T_{\text{tiles}}(x) \\ & + T_{\text{sync}}(x) \\ & + P_{\text{capacity}}(x) \\ & + P_{\text{shape}}(x) \\ & + P_{\text{legality}}(x) \end{aligned}$$

其中：

项	含义	是否是 cycles 语义
T_{tiles}	所有 tile 的主计算/搬运/标量控制成本，考虑并行度	是，analytical cycles
T_{sync}	sync/barrier/event 的估计成本	是，analytical cycles

项	含义	是否是 cycles 语义
P_capacity	UB/L1/L0/GM workspace 容量压力软惩罚	近似 cycles penalty
P_shape	shape/tile 不规整导致的软惩罚	近似 cycles penalty
P_legality	GraphSyncSolver / event reuse / CV pipeline 未验证合法性的风险惩罚	risk penalty, 不等价于真实执行 cycles

T_tiles 定义为：

$$T_tiles(x) = N_tiles / E_parallel * T_tile(x)$$

其中：

N_tiles = 当前 tile shape 下的 tile 数量
E_parallel = effective_parallelism
T_tile = 单 tile 的 corrected steady time

E_parallel 会考虑：

active_blocks = min(block_dim, available_cores, ceil(N_tiles))
waves = ceil(N_tiles / active_blocks)
tail_efficiency = N_tiles / (waves * active_blocks)
E_parallel = active_blocks * clamp(tail_efficiency, 0.20, 1.00)

所以 block_dim 不是越大越好。如果 tile 数很少、tail wave 很碎，有效并行度会被 tail efficiency 拉低。

1.4 4. 单 tile 成本：为什么不是简单 load + compute + store ?

如果没有 double buffer / CV pipeline，模型采用保守串行形式：

T_tile =
T_load_corrected
+ T_compute_corrected
+ T_store_corrected
+ T_workspace_exposed
+ T_scalar_corrected
+ T_schedule

如果开启了 double buffer 或 CV pipeline，load、compute、store 理论上可以部分重叠，所以模型采用 exposed-time 形式：

T_tile =
max(T_load_exposed, T_compute_corrected, T_store_exposed)
+ T_workspace_exposed
+ T_scalar_corrected
+ T_schedule
+ T_warmup_drain

这个设计的含义是：

在稳定流水里，主导时间不是 load + compute + store 全部相加，而是三个 pipeline 中暴露出来的最长路径，加上不能被隐藏的 scalar/control、workspace、schedule 和 warmup/drain。

1.5 5. Base cycles：先算一个没有结构修正的基础估计

模型先根据候选策略和硬件参数计算基础项：

T_load_base
T_store_base
T_cube_base
T_vector_base
T_fix_base
T_sync_base
T_scalar_base
T_workspace_base

这些基础项来自：

base 项	主要来自
T_load_base	tile bytes、dtype bytes、GM/UB/L1/L0 读路径、MTE 带宽估计
T_store_base	tile 输出 bytes、store/fixpipe path、MTE 写路径估计
T_cube_base	tile_m/tile_n/tile_k、cube op proxy、cube 吞吐估计
T_vector_base	vector op count proxy、vector tile workload、vector throughput
T_fix_base	fixpipe/layout conversion/format conversion 的估计
T_sync_base	barrier、set_flag、wait_flag、sync_block 的数量与经验 latency
T_scalar_base	scalar/control/address/index/cast/loop 调度基础开销
T_workspace_base	GM workspace spill/fallback/handoff traffic

注意：这些不是实机测量值，而是 analytical estimate。正因为基础估计会有系统偏差，才需要结构证据修正。

1.6 6. 结构证据到底是什么？

当前模型使用三类输入证据：

1. MLIR 静态结构证据
2. DES 产物结构证据
3. trace event name/count 结构证据

它们共同生成 kernel_cost_profile.json。

1.6.1 6.1 MLIR 静态结构证据

MLIR 侧主要抽取：

证据	例子	说明
flat op counts	mmadL1, copy, nd2nz, fixpipe, vector op, scalar op	判断 kernel 的原始结构组成
scalar family counts	arith_scalar, index_cast, pointer_cast, scf_for, scf_if	判断 scalar/control/address 开销
sync counts	pipe_barrier, set_flag, wait_flag, sync_block_set/wait	判断同步密度
loop-weighted counts	内层循环中的 compute/memory/vector/scalar/sync	内层操作重复执行，权重更高
memory path	GM/UB/L1/L0 之间的静态 path bytes	判断 memory path 压力
buffer lifetime	buffer live span、byte-span pressure	判断 workspace/buffer pressure

证据	例子	说明
alignment/tail	dim/offset misalignment、mask/tail ops	判断 vector/fix/scalar fragmentation
sequence pattern	copy->nd2nz->cube、cube- >fixpipe->vector	判断 pipeline 机会

1.6.2 6.2 DES 产物结构证据

DES 侧主要使用：

证据	使用方式
pipe fraction	判断 lowering 后更像 compute/memory/vector/scalar/sync 哪类 kernel
DMA bytes by space path	增强 memory evidence
sync/barrier ops	增强 sync evidence
critical pipe	作为报告解释字段

重要边界：

当前版本不使用 DES makespan/global duration 做 target calibration。

也就是说，模型不会做：

`predicted_cycles *= DES_makespan / base_prediction`

它只把 DES pipe mix 当作结构比例证据。例如 pipe_s 很高，说明 lowering 后 scalar/control 路径很重，于是主要修正 scalar/control cycles。

1.6.3 6.3 trace event name/count 结构证据

trace 侧只使用 event name 和 count 的结构提示，例如：

`index_cast/cmpi/addi/muli/load/apply -> scalar hint`
`barrier/set_flag/wait_flag/sync -> sync hint`

同样不读取实测 duration target。

1.7 7. 结构比例如何融合？

模型先从 MLIR 得到一组静态比例：

`static_ratios = normalize({compute_score, memory_score, vector_score, scalar_score, sync_score})`

如果 DES 产物可用，再得到一组产物比例：

`product_ratios = normalize({compute_pipe, memory_pipe, vector_pipe, scalar_pipe, sync_pipe})`

当前 V3.3.1 的融合方式为：

`structure_ratios = 0.60 * static_ratios + 0.40 * product_ratios`
`structure_ratios = normalize(structure_ratios)`

为什么不是产物 65% 或 80%？因为 DES pipe fraction 不是实机 profiling。它能反映 lowering 后结构，但仍可能被某一个 pipe 字段支配。降低产物权重可以避免单样本产物过度主导 cost model。

在你上传的样本中，融合后 profile 类似：

```
compute = 0.016
memory  = 0.116
vector  = 0.079
scalar  = 0.553
sync    = 0.236
```

因此该 kernel 被判为：

```
kernel_type = scalar_control_heavy
dominant_component = scalar
```

这不是说真实 cycles 中 scalar 一定占 55.3%，而是说结构证据显示 scalar/control 是最强的修正方向。

1.8 8. 从结构比例到 cycle correction factors

V3.3.1 的核心改动是：结构证据不再到处扩散成一堆 reward/penalty，而是映射为分项 cycle correction factors。

当前主要 factors 为：

```
m_mem      = memory_cycle_correction
m_mem_path  = memory_path_cycle_correction
m_cube     = compute_cycle_correction
m_vec      = vector_cycle_correction
m_align    = alignment_cycle_correction
m_scalar   = scalar_cycle_correction
m_frag     = small_tile_fragmentation_correction
m_sync     = sync_cycle_correction
m_ws       = workspace_pressure_correction
c_overlap  = overlap_confidence
c_cv       = cv_overlap_confidence
```

1.8.1 8.1 每个 factor 的语义

为了避免 PDF 表格过宽，下面逐项说明每个 factor 的含义：

- `memory_cycle_correction`：修正 load/store/fix 的 memory side。主要来自 `memory_ratio`、memory path bytes、buffer pressure。含义是搬运路径比基础估计更重或更轻。
- `memory_path_cycle_correction`：修正 load/store。主要来自 `GM/UB/L1/L0 path bytes`。含义是具体 memory path 的复杂度修正。
- `compute_cycle_correction`：修正 cube compute。主要来自 `compute ratio` 和 `CV opportunity`。它不是 cube reward，只是 cube cycles 的估计修正。
- `vector_cycle_correction`：修正 vector compute。主要来自 `vector ratio`、alignment/tail proxy。
- `alignment_cycle_correction`：修正 vector/fix/scalar fragmentation。主要来自 `dim/offset misalignment` 和 `mask/tail ops`。
- `scalar_cycle_correction`：修正 scalar/control cycles。主要来自 `scalar ratio`、loop-weighted scalar ratio、alignment proxy。
- `small_tile_fragmentation_correction`：修正小 tile 下的 scalar/control 碎片化成本。主要来自 `tile fragmentation` 和 `loop scalar`。
- `sync_cycle_correction`：修正 sync cost。主要来自 `sync ratio` 和 `sync criticality`。
- `workspace_pressure_correction`：修正 workspace exposed cycles。主要来自 `buffer lifetime pressure`。
- `overlap_confidence`：修正 load/store overlap 的可信度。主要来自 `scalar/sync/memory/CV opportunity`，但只做窄范围修正。
- `cv_overlap_confidence`：修正 cube-vector overlap 的可信度。主要来自 `compute/vector balance`、`scalar/sync density`，也只做窄范围修正。

1.8.2 8.2 设计边界

memory evidence 主要进 memory cycles
scalar evidence 主要进 scalar/control cycles
sync evidence 主要进 sync cycles
alignment evidence 主要进 vector/fix/scalar fragmentation
overlap confidence 只做窄范围修正
legality risk 单独进入 P_legality

这个边界是为了避免 double counting。

1.9 9. Corrected component cycles 逐项展开

1.9.1 9.1 Load / Store

$T_{load_corrected} = T_{load_base} * m_mem * m_mem_path$
 $T_{store_corrected} = T_{store_base} * m_mem * m_mem_path$

解释：

基础 load/store 根据 bytes / bandwidth 估计；
如果 MLIR/产物显示 memory path 更复杂、DMA path 更多、buffer pressure 更高，
则通过 m_mem 和 m_mem_path 修正搬运 cycles。

1.9.2 9.2 Cube / Vector / Fix

$T_{cube_corrected} = T_{cube_base} * m_cube$
 $T_{vector_corrected} = T_{vector_base} * m_vec * m_align$
 $T_{fix_corrected} = T_{fix_base} * m_mem * m_align$

解释：

cube 主要受 compute correction 修正；
vector 受 vector correction 和 alignment correction 修正；
fixpipe/layout conversion 同时带有 memory side 和 alignment side，因此使用 m_mem 与 m_align。

1.9.3 9.3 Cube-vector pipeline

$T_{compute_corrected} =$
 $T_{cube_corrected}$
 $+ T_{vector_corrected}$
 $- r_{cv} * c_{cv} * \min(T_{cube_corrected}, T_{vector_corrected})$
 $+ T_{fix_corrected}$

其中：

r_{cv} = CVPipelinePlan 给出的策略 overlap ratio
 c_{cv} = 结构证据给出的 CV overlap confidence

解释：

CV pipeline 的主要收益来自候选策略本身；
结构证据只判断这个收益是否可信，不能把 compute-heavy 直接变成额外 reward。

1.9.4 9.4 Scalar/control

$T_{scalar_corrected} = T_{scalar_base} * m_scalar * m_frag$

解释：

scalar/control 不是主 cube/vector pipeline 的一部分；

大量 index_cast、pointer_cast、scf_for/scf_if、get_block_idx、mask/tail、inner-loop scalar op 会形成不可忽视的调

在你上传的样本里，scalar/control 证据很强，所以 scalar_cycle_correction 明显高于 1。这是合理的，因为该 kernel 的 MLIR 和产物都显示 scalar/control-heavy。

1.9.5 9.5 Workspace

$T_{\text{workspace_exposed}} = T_{\text{workspace_base}} * m_{\text{ws}}$

解释：

GM workspace / handoff / spill traffic 不是普通 load/store 的完全可隐藏部分；
它更像 fallback traffic，因此作为 exposed cost 叠加。

1.9.6 9.6 Sync

$T_{\text{sync}} = T_{\text{sync_base}} * m_{\text{sync}}$

其中基础 sync 通常来自：

$T_{\text{sync_base}} =$
 $\text{num_barrier} * \text{barrier_latency}$
 $+ \text{num_set_flag} * \text{set_flag_latency}$
 $+ \text{num_wait_flag} * \text{wait_flag_latency}$
 $+ \text{sync_block_cost}$

解释：

sync evidence 只修正同步操作本身的估计开销。

GraphSyncSolver 未验证是否合法，不应该混进 T_{sync} ，而应该放到 P_{legality} 。

1.10 10. Overlap 如何处理？

1.10.1 10.1 Load/store overlap

候选策略先给出基础 overlap：

$r_{\text{load_strategy}}$
 $r_{\text{store_strategy}}$

结构证据只给出 confidence：

$r_{\text{load}} = r_{\text{load_strategy}} * c_{\text{overlap}}$
 $r_{\text{store}} = r_{\text{store_strategy}} * c_{\text{overlap}}$

然后：

$T_{\text{load_exposed}} = T_{\text{load_corrected}} * (1 - r_{\text{load}})$
 $T_{\text{store_exposed}} = T_{\text{store_corrected}} * (1 - r_{\text{store}})$

1.10.2 10.2 为什么 overlap confidence 是窄范围？

因为 overlap 的主因应该是策略和硬件合法性，例如：

double_buffer 是否开启
per-buffer multiplier 是否足够
stage buffer 是否放得下
producer-consumer distance 是否合理
UB/L1/L0 是否 overflow

结构证据只能说“这个 overlap 可信度更高或更低”，不能说“scalar-heavy 所有 overlap 全部大幅失效”。因此 V3.3.1 将 overlap confidence 限定在较窄范围，避免重复惩罚。

1.11 11. Penalty 的语义

1.11.1 11.1 Capacity penalty

`P_capacity = memory_pressure_penalty(scope_utilization, hw)`

它表示：虽然没有触发硬件 gate 的 hard overflow，但 UB/L1/L0 等资源已经接近边界，可能导致更高风险或更差调度。

1.11.2 11.2 Shape penalty

`P_shape = shape_regularization_penalty(tile shape, kernel shape, hw)`

它表示：tile shape 与 kernel shape/hardware granularity 不匹配，可能产生 tail、fragmentation 或不规则访问。

1.11.3 11.3 Legality risk penalty

`P_legality =`
 `P_graph_sync_unknown`
 `+ P_event_reuse_unknown`
 `+ P_cv_pipeline_estimated`

它专门表达策略合法性/可落地性不确定性。例如：

`GraphSyncSolver status = UNKNOWN`
`CV pipeline legality = PASS_ESTIMATED`

这类风险不能说是真实执行时间，因此单独放在 `P_legality`，不污染 `T_sync` 的 cycles 语义。

1.12 12. V3.3.1 如何利用结构化信息？

完整链路如下：

```
MLIR + product artifacts
  |
  v
extract structural evidence
  |
  v
compute static_ratios and product_ratios
  |
  v
fuse ratios: 60% MLIR + 40% artifact
  |
  v
build kernel_cost_profile
  |
  v
map evidence to cycle correction factors
  |
  v
correct load/compute/store/scalar/sync/workspace cycles
  |
  v
```

apply strategy overlap and narrow confidence correction

|
v

sum corrected cycles + necessary penalties

|
v

predicted_cycles

也就是：结构化信息现在不是独立 score，而是进入每个分项 cycle estimate。

1.12.1 12.1 例子：一个 scalar-heavy kernel

如果 MLIR/产物显示：

scalar_ratio 高
loop_weighted_scalar 高
pipe_s fraction 高
sync density 较高

V3.3.1 的行为是：

scalar_cycle_correction 上升
small_tile_fragmentation_correction 上升
sync_cycle_correction 适度上升
overlap_confidence 轻微下降

但它不会再让同一个 scalar-heavy 证据同时大幅：

提高 scalar cost
提高 sync cost
提高 small tile penalty
大幅压低 overlap reward
大幅压低 CV reward
提高 legality risk

这就是 V3.3.1 相比 V3.3 更干净的地方。

1.13 13. 和 V3.3 的区别

1.13.1 13.1 V3.3 的逻辑

V3.3 更像：

结构证据 -> 多个 multiplier/reward/penalty -> total predicted cycles

当 kernel 被判为 scalar/sync-heavy 时，同一类证据可能同时影响：

scalar_control_multiplier
small_tile_scalar_penalty_scale
loop_weighted_scalar_multiplier
sync_multiplier
sync_criticality_multiplier
overlap_reward_scale
cv_reward_scale
cube_reward_scale
legality_risk_penalty

问题是：这些路径之间存在语义重叠。比如 scalar-heavy 既提高 scalar_control_time，又降低 overlap，又降低 CV reward，还可能提高 sync/risk。最终 predicted_cycles 仍然叫 cycles，但内部更像“多重加权 ranking score”。

1.13.2 13.2 V3.3.1 的逻辑

V3.3.1 改成：

结构证据 -> 对应分项 cycle correction -> total predicted cycles

核心变化：

方面	V3.3	V3.3.1
总体语义	structure-aware weighted cost	structure-aware corrected cycle estimate
结构证据用途	可同时影响多个 cost/reward/penalty	每类证据主要修正对应分项 cycles
scalar evidence	scalar、small tile、sync、overlap、CV reward 多路径影响	主要进入 scalar/control 和 fragmentation
sync evidence	sync multiplier、criticality multiplier、risk 可能叠加	sync cycles 与 legality risk 分离
overlap	可能被 scalar/sync 较强打折	只做窄范围 confidence 修正
DES artifact 权重	产物比例更强，容易主导	MLIR 60% + artifact 40% 保守融合
predicted_cycles 语义	更像 cycles-shaped ranking score	更像 corrected component cycles sum
double counting 风险	较高	明显降低

1.13.3 13.3 为什么 V3.3.1 更适合汇报？

因为可以清楚回答领导的几个问题：

- 问：你的目标到底是算 score 还是 cycles？答：目标是估计 total predicted cycles。
- 问：结构信息干什么用？答：修正 load/compute/store/scalar/sync/workspace 各分项 cycles 的基础估计误差。
- 问：有没有用实机数据硬拟合？答：没有。在线寻优不使用 msprof target，不使用 DES makespan/global scale。
- 问：为什么不是 load+compute+store+sync 直接加？答：因为有 pipeline overlap。主 tile 部分在流水模式下使用 max(exposed load, compute, exposed store)，然后再加 scalar/control、workspace、schedule 等不可隐藏项。
- 问：GraphSyncSolver 不确定怎么办？答：执行时间估计和合法性风险分开；未知合法性进入 P_legality，不污染 sync cycles。

1.14 14. 当前样本的解释方式

以 kernel_001.npuir.mlir + prefill_des.json + prefill_trace.json 为例，当前 profile 显示：

dominant_component = scalar
kernel_type = scalar_control_heavy

融合后的结构比例大致为：

compute: 0.016
memory : 0.116
vector : 0.079
scalar : 0.553
sync : 0.236

对应 correction factors 大致为：

memory_cycle_correction = 1.10
compute_cycle_correction = 1.06
vector_cycle_correction = 1.02

```
scalar_cycle_correction = 1.72
sync_cycle_correction = 1.36
overlap_confidence = 0.88
cv_overlap_confidence = 0.90
```

可以这样解释：

这个 kernel 不是典型 cube-heavy kernel，而是 scalar/control 与 sync 结构比较重。
因此模型主要提高 scalar/control cycles 和 sync cycles 的估计；
memory/compute/vector 只做轻微修正；
overlap 与 CV pipeline 收益只被轻度降置信，而不是被结构证据大幅砍掉。
这比 V3.3 的解释更稳，因为不会出现“一个 scalar-heavy 判断到处乘”的问题。

1.15 15. 当前模型仍然不能证明什么？

当前模型可以用于：

- 候选策略排序
- 缺陷 MLIR 的相对诊断
- 硬件边界检查
- 解释为什么某些策略被惩罚或被选中
- 在没有实机 profiling 时做 conservative analytical search

但不能直接证明：

- 真实 msprof cycles 一定等于 predicted_cycles
- 真实 speedup 一定等于 predicted speedup
- GraphSyncSolver 策略一定可 rewrite 且 deadlock-free
- DES pipe fraction 与实机 pipe active/stall cycles 完全一致

如果要把它升级为更真实的 hardware performance model，需要：

- 同一批 kernel 的 current IR 与候选策略 rewrite 后实机 msprof 数据
- 每个候选的编译产物 / DES 产物，而不是共享一个 kernel-level artifact profile
- 用实机数据离线拟合 correction factors 的 alpha/clamp/config
- 用 held-out kernel 验证 ranking accuracy 和 cycles prediction error

1.16 16. 推荐的口头表述

汇报时可以这样说：

我们当前的 cost model 目标仍然是估计总 cycles，但不是直接使用实机 profiling。

模型先根据 tile、buffer、pipeline、sync plan 和硬件配置，计算 load、compute、store、scalar/control、sync 等基础 cycles。

然后利用 MLIR 和编译产物中的结构证据生成 kernel-level cycle correction factors。这些结构证据包括 op count、loop-weighted scalar/sync、memory path、buffer lifetime、DES pipe mix 和 trace event name count。

这些 correction factors 只修正对应的分项 cycles。例如 memory 证据修正 load/store，scalar 证据修正 scalar/control，sync 证据修正 sync。

最后模型汇总 corrected component cycles，并额外加入容量、shape 和合法性风险惩罚，得到 predicted_cycles。

相比 V3.3，V3.3.1 最大变化是降低了结构证据的重复加权风险。结构信息不再同时作为多路 reward/penalty 扩散，而是回到分项 cycles 修正的语义上。

1.17 17. 文件和报告中应该怎么看

运行后重点看这些文件：

文件	看什么
kernel_cost_profile.json	kernel 类型、结构比例、cycle correction factors
cost_breakdown.json	selected strategy 的分项 cycles、overlap saving、penalty
selected_strategy.json	最终选择的候选参数
selected_plan.json	四类 plan 的具体取值与 legality 状态
top_candidates.json	Top 候选的排序与差异
hardware_boundary_audit.json	UB/L1/L0/GM workspace 是否接近或超过边界

最关键的是：不要只看 predicted_cycles 一个数字，要同时看：

load_exposed
store_exposed
cube_vector_time
scalar_control_time
sync_cost
memory_pressure_penalty
shape_regularization_penalty
legality_risk_penalty
kernel_cost_profile_weights

这些分项能说明为什么候选被选中或被压下去。

1.18 18. 小结

V3.3.1 的 cost model 可以概括为：

structure-aware corrected component cycle model

它的关键优点是：

1. 目标仍然是 total predicted cycles；
2. 结构证据用于修正分项 cycles，而不是额外打分；
3. memory/scalar/sync/vector/compute 各有相对清晰的修正边界；
4. overlap 由策略主导，结构证据只做窄范围 confidence 修正；
5. legality risk 与 sync cycles 分离；
6. 相比 V3.3, double counting 风险更低，汇报语义更清楚。

一句话：

V3.3.1 不是把结构信息拿来“再加权一次总成本”。它的做法是：

结构信息 -> 修正每个分项 cycles 的估计误差 -> 汇总为 predicted total cycles